Reporte Práctica 10: Árboles Binarios Ordenados

Emmanuel Cruz Durán

3 de noviembre de 2026

1. Complicaciones en el desarrollo

La programación de esta práctica presentó dos obstáculos interesantes. El más demandante a nivel algorítmico fue la eliminación de vértices con dos hijos. Mover punteros suena sencillo en papel o en el pizarrón. Sin embargo, al pasarlo a código es sumamente fácil romper las referencias del padre o dejar al nodo sucesor volando en la memoria sin conexión. Tuve que arrastrar el lápiz y hacer pruebas de escritorio con varios casos para asegurarme de que aislar el elemento mínimo del subárbol derecho no dejara nodos huérfanos ni alterara la propiedad de búsqueda del árbol.

Más allá de la lógica de las estructuras de datos, el procesamiento del archivo de inventario trajo su propia complicación. El archivo de texto venía con un formato dañado en la primera línea. Faltaba una coma divisoria en los datos del iPhone, lo que provocaba que la función de separación fallara y el programa colapsara al intentar convertir texto a números flotantes. Resolví esto metiendo un bloque condicional específico en el método de carga que atrapa esa línea anómala y la reconstruye con los campos correctos antes de instanciar el objeto. Es un buen recordatorio de que los algoritmos rara vez fallan solos; los datos de entrada sucios suelen ser el verdadero problema.

2. Árboles Rojinegros y AVL

Un árbol binario ordenado convencional carece de mecanismos de balanceo; si los datos se insertan secuencialmente, el árbol se degenera en una lista ligada, degradando la complejidad de búsqueda de $O(\log n)$ a $O(n)$.

- **Árboles AVL:** Son árboles binarios de búsqueda auto-balanceables. Mantienen la propiedad estricta de que la diferencia de alturas entre el subárbol izquierdo y derecho de cualquier nodo es como máximo 1. Si la propiedad se viola tras una inserción o eliminación, aplican rotaciones para restaurar el equilibrio.
- **Árboles Rojinegros:** Relajan la condición estricta de altura utilizando un bit de çolor"(rojo o negro) en cada nodo. Mediante reglas cromáticas aseguran que el camino más largo desde la raíz a una hoja no exceda el doble del camino más corto. Ofrecen inserciones y eliminaciones ligeramente más rápidas que los AVL al requerir menos rotaciones estructurales en promedio.

Ambas estructuras garantizan que la altura del árbol siempre esté acotada por $O(\log n)$, solucionando la ineficiencia de las búsquedas en árboles desbalanceados. *Fuente: Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.*

3. Relación de orden y Preguntas

La relación de orden impuesta por el método `__lt__` de la clase `Producto` se basa en el ordenamiento lexicográfico del atributo `nombre`. Se decidió esta relación dado que el identificador natural para realizar consultas en un inventario de cara al usuario final es el nombre del producto, lo cual garantiza búsquedas directas en tiempo logarítmico sobre el árbol.

a) ¿Los árboles binarios ordenados son mejores que los árboles completos?
Depende, no simplemente son mejores, tienen propósitos distintos. Un árbol binario completo asegura propiedades geométricas y un almacenamiento contiguo sin huecos, siendo la base ideal para implementar Heaps y colas de prioridad. Un árbol binario ordenado establece una relación jerárquica de precedencia entre nodos, lo cual es inútil para un Heap, pero indispensable para bases de datos, inventarios y búsquedas eficientes.

b) ¿Es posible determinar cuáles productos están en un rango de precios sin usar el método devolver_recorrido()?
Sí. Es posible implementar un método de recorrido interno ad-hoc (por ejemplo, un DFS iterativo empleando una pila directamente en la clase `Inventario`) que evalúe y filtre el precio de cada producto sobre la marcha, guardando en un arreglo únicamente las coincidencias. Esto evita el alto costo en memoria $O(n)$ de instanciar la estructura pesada `ListaDoblementeLigada` entera antes de hacer el filtrado lógico.

Respuestas de ChatGPT

a) No necesariamente. Los árboles binarios ordenados y los árboles binarios completos tienen ventajas distintas, dependiendo del problema. Los árboles binarios ordenados permiten buscar, insertar y eliminar elementos eficientemente si están balanceados, mientras que los árboles binarios completos aprovechan mejor el espacio y son útiles para representar estructuras como los montículos. Por lo tanto, ninguno es universalmente mejor que el otro; la elección depende de las operaciones que se necesiten realizar.

b) Sí. Es posible determinar qué productos están dentro de un rango de precios sin utilizar el método `devolver_recorrido()`. Si el árbol está ordenado por precio, se puede realizar un recorrido que visite únicamente los nodos cuyos precios estén dentro del rango solicitado. Por ejemplo, si el precio de un nodo es menor que el límite inferior, se puede descartar su subárbol izquierdo; si es mayor que el límite superior, se puede descartar su subárbol derecho. Esto permite evitar visitar nodos que no contienen productos relevantes y hacer la búsqueda más eficiente.

4. Análisis Crítico

Al comparar nuestras conclusiones sobre el primer inciso (si un árbol ordenado es mejor que uno completo), ambos coincidimos en que la pregunta plantea un falso dilema. No existe una estructura universalmente superior. La IA acertó al mencionar que los árboles completos aprovechan mejor el espacio y son la base de los montículos, lo cual va exactamente en la misma línea de mi argumento geométrico. En esta pregunta, la herramienta dio una respuesta sólida, bien planteada y muy alineada con lo que vimos en clase.

En la segunda pregunta, Chat propuso optimizar la búsqueda por rangos de precio descartando ramas enteras del árbol, sugiriendo que si un nodo supera el límite del precio podemos ignorar su

subárbol. Suena muy lógico, pero es un error. Obviamente, a la IA le falta contexto de mi práctica, ignoró por completo que nuestro criterio de ordenamiento en la clase `Producto` es alfabético, no numérico. Si hiciéramos lo que sugiere la IA, terminaríamos podando partes de la estructura que muy probablemente contienen productos con precios válidos, arruinando los resultados de la consulta. Es interesante ver que la mayoría de errores que comete la IA, es por falta de contexto o memoria, se que es ajeno a esta práctica, pero veo ahora porque es tan popular lo del "harness engineering".